



HARDWARE ARCHITECTURE & QLORA OPTIMIZATION

This document details the VRAM footprint, memory spike prevention architecture, and the optimized code required to fine-tune the Gemma 4B SLM on an NVIDIA RTX 4070 Super (12GB VRAM) across 12M+ records efficiently.

I. VRAM Footprint on RTX 4070 Super (12GB)

With a sequence length capped at **384 tokens** and a batch size of **4**, training a 4B parameter model in 4-bit quantization requires approximately **6.0 GB to 7.0 GB of VRAM**.

COMPONENT	ESTIMATED VRAM USAGE	NOTES
4-bit Base Model Weights	~2.2 GB	Loaded via NF4 (bitsandbytes or Unsloth)
LoRA Trainable Adapters	~0.3 GB	Rank 16, Alpha 32 across target modules
8-bit AdamW Optimizer States	~0.2 GB	Tracks trainable LoRA weights, not base weights
Activations (Batch=4, Len=384)	~2.5 GB	Managed by Gradient Checkpointing
PyTorch CUDA Overhead & Context	~1.2 GB	Driver and PyTorch workspace memory
TOTAL ESTIMATED VRAM	~6.4 GB – 6.8 GB	Leaves ~5.2 GB of free VRAM headroom

II. Memory Spike Prevention Architecture

Spike Cause 1: Variable-Length Sequences

The Problem: Mixed sequence lengths cause memory requirements to jump unpredictably.
The Solution: Enabling **Sequence Packing** in SFTTrainer packs short sequences into fixed 384-token blocks. Every batch entering the GPU is exactly the same shape, keeping memory usage completely flat and predictable.

Spike Cause 2: Activation Memory Explosion

The Problem: PyTorch stores intermediate activations in VRAM during the forward pass, which accumulate exponentially.
The Solution: **Gradient Checkpointing** discards intermediate activations and recalculates them on-the-fly during the backward pass, eliminating activation spikes.

Spike Cause 3: Memory Fragmentation

The Problem: Repeatedly allocating and freeing tensors of different sizes leaves fragmented "holes" in GPU memory.

The Solution: Setting `PYTORCH_CUDA_ALLOC_CONF="expandable_segments:True"` instructs PyTorch to expand existing memory segments rather than splitting them, maintaining contiguous memory.

Spike Cause 4: Accumulation Buffers

The Problem: Setting high per-device batch sizes causes memory to scale linearly.

The Solution: Keeping `per_device_train_batch_size=4` and using `gradient_accumulation_steps=4` yields an effective batch size of 16 without storing 16 sequences simultaneously.

III. Hardware Optimization (Ada Lovelace)

The RTX 4070 Super features 4th Generation Tensor Cores (Ada Lovelace architecture). To maximize performance:

- **Use bfloat16:** Ada Lovelace natively supports BF16, preventing loss scale spikes and gradient underflows/overflows.
- **Install Unsloth:** Unsloth uses custom Triton kernels that can drop total VRAM consumption on a 4070 Super down to ~4.5 GB while doubling token throughput.
- **Data Loader Workers:** Set `dataloader_num_workers=4` to ensure the GPU's CUDA cores are continually fed without waiting on the CPU.

IV. Optimized QLoRA Script (optimized_qlora.py)

This script transitions to the TRL SFTTrainer to enable sequence packing and native Unsloth support, eliminating GPU stalling.

```
import os
os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "expandable_segments:True"
import sys
import argparse
import time
import torch
import gc
from datasets import load_dataset
import yaml

# TRL is the standard for high-performance alignment
from trl import SFTTrainer, SFTConfig
from transformers import (
    AutoTokenizer,
    BitsAndBytesConfig,
    TrainerCallback,
)

# Unsloth is highly recommended for Gemma 4 models (2x faster, 70% less VRAM)
try:
    from unsloth import FastLanguageModel
    HAS_UNSLOTH = True
except ImportError:
    HAS_UNSLOTH = False
    from transformers import Gemma4ForConditionalGeneration
    from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training

class CustomControlCallback(TrainerCallback):
    """
    Optimized Callback: No longer checks loss on EVERY step (which ruins GPU sync).
    It only checks during predefined logging steps.
    """
    def __init__(self, stop_checker=None, progress_callback=None, max_minutes=0,
total_dataset_records=15000000):
        self.stop_checker = stop_checker
        self.progress_callback = progress_callback
        self.max_minutes = max_minutes
        self.total_dataset_records = total_dataset_records
        self.start_time = time.time()
        self.last_heartbeat_time = time.time()
        self.last_step = 0
        self.latest_loss = 0.0

    def on_log(self, args, state, control, logs=None, **kwargs):
        # Only extract loss during standard logging events to prevent GPU sync stalls
        if logs and "loss" in logs:
            self.latest_loss = float(logs["loss"])

    def on_step_end(self, args, state, control, **kwargs):
        if self.stop_checker and self.stop_checker():
            print("\n[QLoRA] Stop requested by user! Halting training gracefully...")
            control.should_training_stop = True

        if self.max_minutes > 0:
            elapsed_mins = (time.time() - self.start_time) / 60.0
```

```

        if elapsed_mins >= self.max_minutes:
            print(f"\n[QLoRA] Max time limit reached ({self.max_minutes} mins)! Halting...")
            control.should_training_stop = True

    now = time.time()
    if now - self.last_heartbeat_time >= 60.0:
        self.last_heartbeat_time = now
        steps_done = state.global_step - self.last_step
        self.last_step = state.global_step
        sec_per_it = 60.0 / max(1, steps_done)
        pct_run = (state.global_step / max(1, state.max_steps)) * 100.0 if state.max_steps > 0
    else 0.0

    eff_batch = (args.per_device_train_batch_size or 1) * (args.gradient_accumulation_steps or
1)

    records_processed = state.global_step * eff_batch

    elapsed_sec = int(now - self.start_time)
    m, s = divmod(elapsed_sec, 60)
    h, m = divmod(m, 60)
    time_str = f"{h:02d}:{m:02d}:{s:02d}"

    vram_str = ""
    if torch.cuda.is_available():
        allocated = round(torch.cuda.memory_allocated(0) / (1024 ** 3), 2)
        vram_str = f" | GPU VRAM={allocated} GB"

    print(f"[Heartbeat] Step {state.global_step}/{state.max_steps} ({pct_run:.1f}%) | "
          f"Processed: ~{records_processed:,} sequences | Loss: {self.latest_loss:.4f} | "
          f"Pace: {sec_per_it:.1f}s/it | Elapsed: {time_str}{vram_str}")

def run_qlora_training(
    config_path: str = "config.yaml",
    dataset_path: str = "Finetune/training_data.jsonl",
    output_dir: str = "Finetune/output_adapter",
    epochs: int = 1,
    batch_size: int = 4, # Target Effective Batch size of 16 (e.g., batch 4 * accum 4)
    learning_rate: float = 2e-4, # Recommended start for normal QLoRA
    max_steps: int = 4500,
    sub_sample: int = 45000,
    max_length: int = 384,
    r: int = 16, # Suggested ranks for Unsloth/QLoRA: 8, 16, or 32
    lora_alpha: int = 32,
    dataloader_num_workers: int = 4, # Increased to prevent CPU bottlenecking
    max_minutes: int = 0,
    mixed_precision: str = "Auto",
    grad_accum: int = 4,
    model_path: str = None,
):
    print("\n" + "=" * 60)
    print("  High-Performance QLoRA + TRL Sequence Packing Pipeline")
    print("=" * 60)

    # 1. Load Configuration
    base_model_path = model_path if model_path else "d:/models/HyrLink-Core-SLM-v1"
    if not model_path and os.path.exists(config_path):
        with open(config_path, "r") as f:
            cfg = yaml.safe_load(f)
            base_model_path = cfg.get("model", {}).get("base_path", base_model_path)

    is_cuda = torch.cuda.is_available()
    compute_dtype = torch.float16

```

```

    if mixed_precision == "BF16" or (mixed_precision == "Auto" and is_cuda and
torch.cuda.is_bf16_supported()):
        compute_dtype = torch.bfloat16

# 2. Model Loading (Routing to Unsloth if installed, otherwise optimized HF)
print(f"[QLoRA] Loading Model: {base_model_path}")
if HAS_UNSLOTH:
    print("[QLoRA] Unsloth detected! Using custom Triton kernels for 2x speedup.")
    model, tokenizer = FastLanguageModel.from_pretrained(
        model_name=base_model_path,
        max_seq_length=max_length,
        dtype=compute_dtype,
        load_in_4bit=True,
    )
    model = FastLanguageModel.get_peft_model(
        model,
        r=r,
        target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj",
"down_proj"],
        lora_alpha=lora_alpha,
        lora_dropout=0, # Unsloth optimizes internal code if dropout=0
        bias="none",
        use_gradient_checkpointing="unsloth", # 70% VRAM reduction
    )
else:
    print("[QLoRA] Unsloth not detected. Using native HuggingFace Transformers.")
    tokenizer = AutoTokenizer.from_pretrained(base_model_path)
    tokenizer.padding_side = "right"
    if not tokenizer.pad_token:
        tokenizer.pad_token = tokenizer.eos_token

    bnb_config = BitsAndBytesConfig(
        load_in_4bit=True,
        bnb_4bit_use_double_quant=True,
        bnb_4bit_quant_type="nf4",
        bnb_4bit_compute_dtype=compute_dtype,
    )
    model = Gemma4ForConditionalGeneration.from_pretrained(
        base_model_path,
        quantization_config=bnb_config,
        device_map="auto",
        attn_implementation="sdpa", # Use Scaled Dot-Product Attention
        torch_dtype=compute_dtype
    )
    model = prepare_model_for_kbit_training(model, use_gradient_checkpointing=True)
    lora_config = LoraConfig(
        r=r,
        lora_alpha=lora_alpha,
        target_modules=r".*language_model\.layers\.d+\.self_attn\.(q|k|v|o)_proj|mlp\.(
gate|up|down)_proj$",
        lora_dropout=0.05,
        bias="none",
        task_type="CAUSAL_LM",
    )
    model = get_peft_model(model, lora_config)

# 3. Load & Format Dataset (On the fly, saves RAM)
dataset = load_dataset("json", data_files=dataset_path, split="train")
if sub_sample > 0 and len(dataset) > sub_sample:
    dataset = dataset.shuffle(seed=42).select(range(sub_sample))

def formatting_prompts_func(example):
    messages = example.get("messages", [])

```

```

        if getattr(tokenizer, "chat_template", None) is not None:
            text = tokenizer.apply_chat_template(messages, tokenize=False,
add_generation_prompt=False)
        else:
            text = "\n".join([f"{m.get('role', 'user')}: {m.get('content', '')}" for m in messages])
        return text

# 4. Optimized Training Arguments
# Removed logging_steps=1 to stop GPU sync throttling.
training_args = SFTConfig(
    output_dir=output_dir,
    per_device_train_batch_size=batch_size,
    gradient_accumulation_steps=grad_accum,
    learning_rate=learning_rate,
    lr_scheduler_type="cosine",
    warmup_steps=100,
    max_steps=max_steps if max_steps > 0 else -1,
    num_train_epochs=epochs if max_steps <= 0 else 100,
    logging_steps=20, # <--- FIX: Increased from 1 to 20 for maximum throughput
    save_strategy="steps" if max_steps > 0 else "epoch",
    save_steps=500,
    save_total_limit=2,
    dataloader_num_workers=dataloader_num_workers, # <--- FIX: Re-enabled multi-threading
    dataloader_pin_memory=is_cuda,
    packing=True, # <--- FIX: Eliminates padding tokens for 100% compute efficiency
    max_seq_length=max_length,
    dataset_text_field="text",
    bf16=True if compute_dtype == torch.bfloat16 else False,
    fp16=True if compute_dtype == torch.float16 else False,
    optim="adamw_8bit",
    report_to="none",
)

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=dataset,
    formatting_func=formatting_prompts_func, # Formats dynamically during training
    args=training_args,
    callbacks=[CustomControlCallback(stop_checker_fn, progress_callback_fn, max_minutes)],
)

print("[QLoRA] Starting high-performance training run...")
trainer.train()

print(f"[QLoRA] Saving fine-tuned adapter to {output_dir}...")
model.save_pretrained(output_dir)
tokenizer.save_pretrained(output_dir)
print("[QLoRA] Fine-tuning complete!")

if __name__ == "__main__":
    run_qlora_training()

```